Single-Agent system vs Multi-Agent system

Date: October 28, 2025

Abstract:

This document provides a comprehensive comparison between Single-Agent Systems (SAS) and Multi-Agent Systems (MAS). It covers theoretical foundations, architectures, workflows, algorithms, communication and coordination strategies, metrics for comparison, practical applications across industries, and case studies. The goal is to produce a thesis-style document suitable for academic review or to serve as a foundation for further research and implementation.

Keywords: Single-agent system, multi-agent system, MAS, SAS, coordination, communication, distributed AI, swarm robotics, autonomous systems, architectures, workflows.

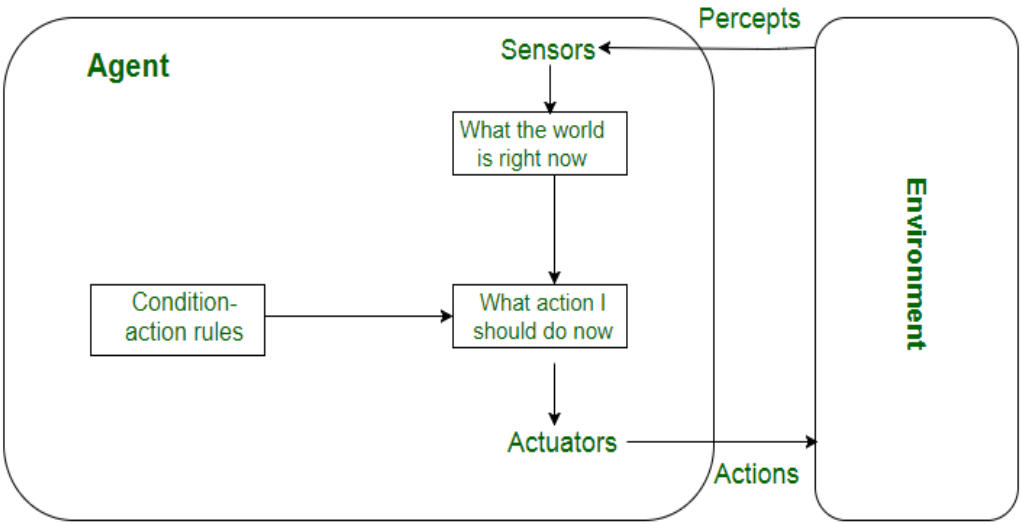
INDEX

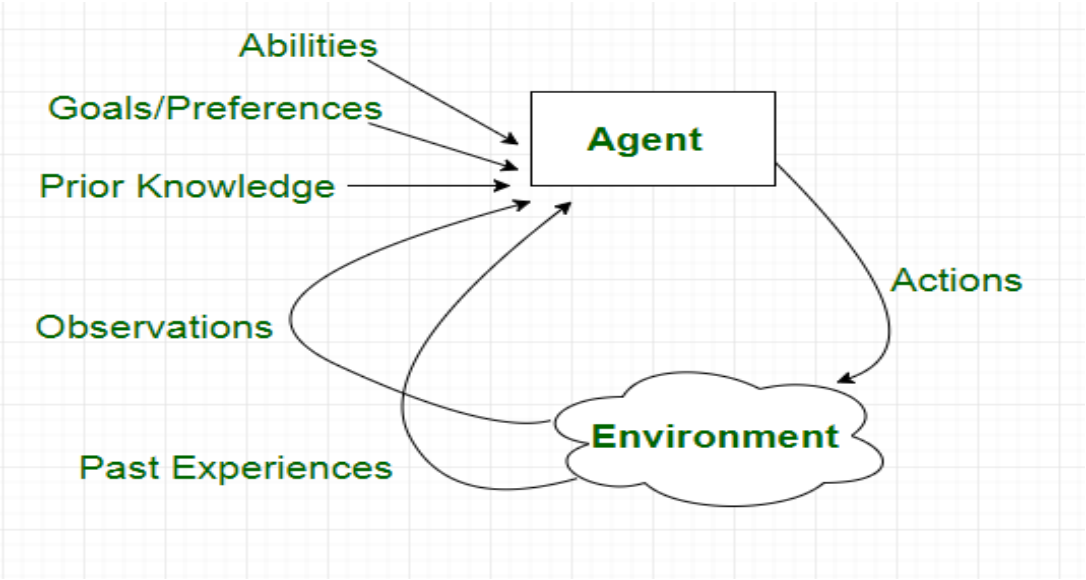
Table of Contents	Page
Introduction	1
Background and Definitions	2
Theoretical Foundations	2
Single-Agent Systems(SAS)	3
Multiple-Agent Systems(MAS)	4
Comparative Analysis	5
Real-World Applications and Case Studies	6
AI Agent Stack	
Implementation	7
Ethical, Security and Practical Considerations	7
AI Agents through Industry POV	8
Conclusion and Future Directions	9
Appendices	10

Introduction

Agents:

An autonomous computational entities that perceive their environment and act to achieve goals are central to modern AI systems. This thesis explores the differences between designs that rely on a single agent (Single-Agent Systems, SAS) and those that use multiple interacting agents (Multi-Agent Systems, MAS). Understanding both approaches, their trade-offs, and their real-world applications allows designers to choose the best paradigm for a task.





Background and Definitions

2.1. Agent:

An agent is an entity with the ability to perceive its environment through sensors, act upon that environment via actuators, and pursue goals using decision-making capabilities. Agents can be simple reflex systems or complex deliberative systems.

2.2. Environment types:

Environments are characterized along dimensions: deterministic vs. stochastic, static vs. dynamic, episodic vs. sequential, discrete vs. continuous, and single-agent vs. multi-agent.

2.3. Autonomy, reactivity, pro-activeness, social ability:

Key agent properties include autonomy (operates without human intervention), reactivity (responds to environmental changes), pro-activeness (goal-directed behavior), and social ability (interacts with other

Theoretical Foundations

3.1. Intelligent agents in AI:

Covers rationality, utility functions, and decision-theoretic agents. Discusses architectures such as model-based, goal-based, and utility-based agents.

3.2. Game theory and MAS :

MAS often leverage game-theoretic concepts: Nash equilibria, cooperative vs. non-cooperative games, mechanism design, and repeated games for strategy learning.

3.3. Distributed systems principles:

Topics: consensus (Paxos/Raft), fault tolerance, partial knowledge, message passing vs. shared memory, eventual consistency.

Single-Agent Systems (SAS)

4.1. Architecture and internal workflows:

A typical SAS pipeline:

Perception: sensor input, preprocessing, feature extraction.

State Estimation: filters (Kalman, particle filter).

Planning/Decision Making: search, optimization, RL policies.

Execution & Control: trajectory control, actuators.

Monitoring & Learning: online adaptation, logging.

4.2. Algorithms and design patterns:

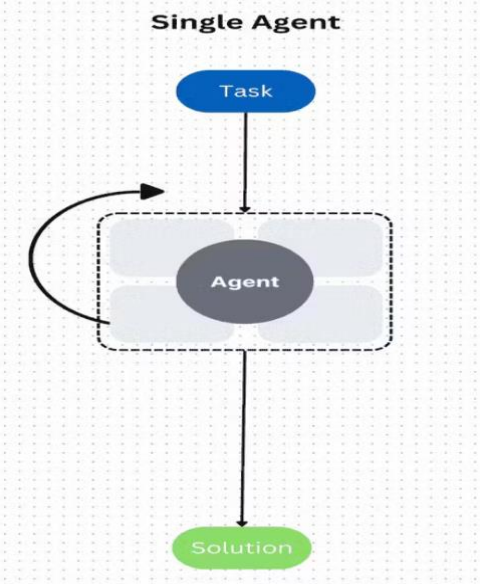
Search algorithms (A*, Dijkstra), optimal control, PID controllers, model predictive control (MPC), classical RL (Q-learning), and modern deep RL methods.

4.3. Strengths and limitations:

Strengths: simpler design, easier verification, lower communication overhead.
Limitations: single point of failure, limited scalability, narrow scope.

4.4. Representative use-cases

Single-robot manipulation, personal assistants, single autonomous vehicle control, recommendation engines for single-user personalization.



Multi-Agent Systems (MAS)

5.1. MAS architectures:

Centralized: a central planner/coordination node; agents act on commands. Easier to optimize globally but single point of failure.

Decentralized: no central controller; agents coordinate locally, enabling resilience and scalability.

Hybrid: combines both (local autonomy with periodic centralized planning).

5.2. Communication & protocols:

Agents communicate via message passing, shared blackboards, publish/subscribe patterns. Common standards include ACL (Agent Communication Language) and FIPA specifications.

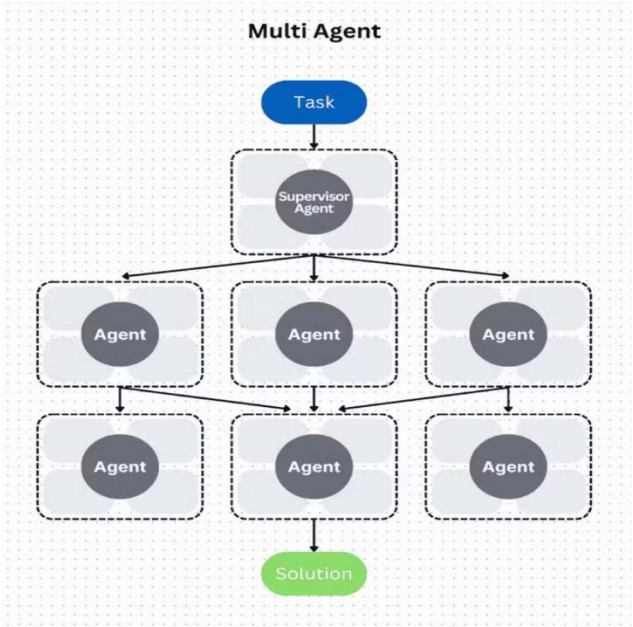
5.3. Coordination mechanisms:

Negotiation and contracts: bilateral or multilateral negotiation.

Market-based/auctions: resources allocated via bidding.

Consensus algorithms: distributed agreement for critical decisions.

Stigmergy: indirect coordination via environment modifications (common in swarm robotics).



Comparative Analysis

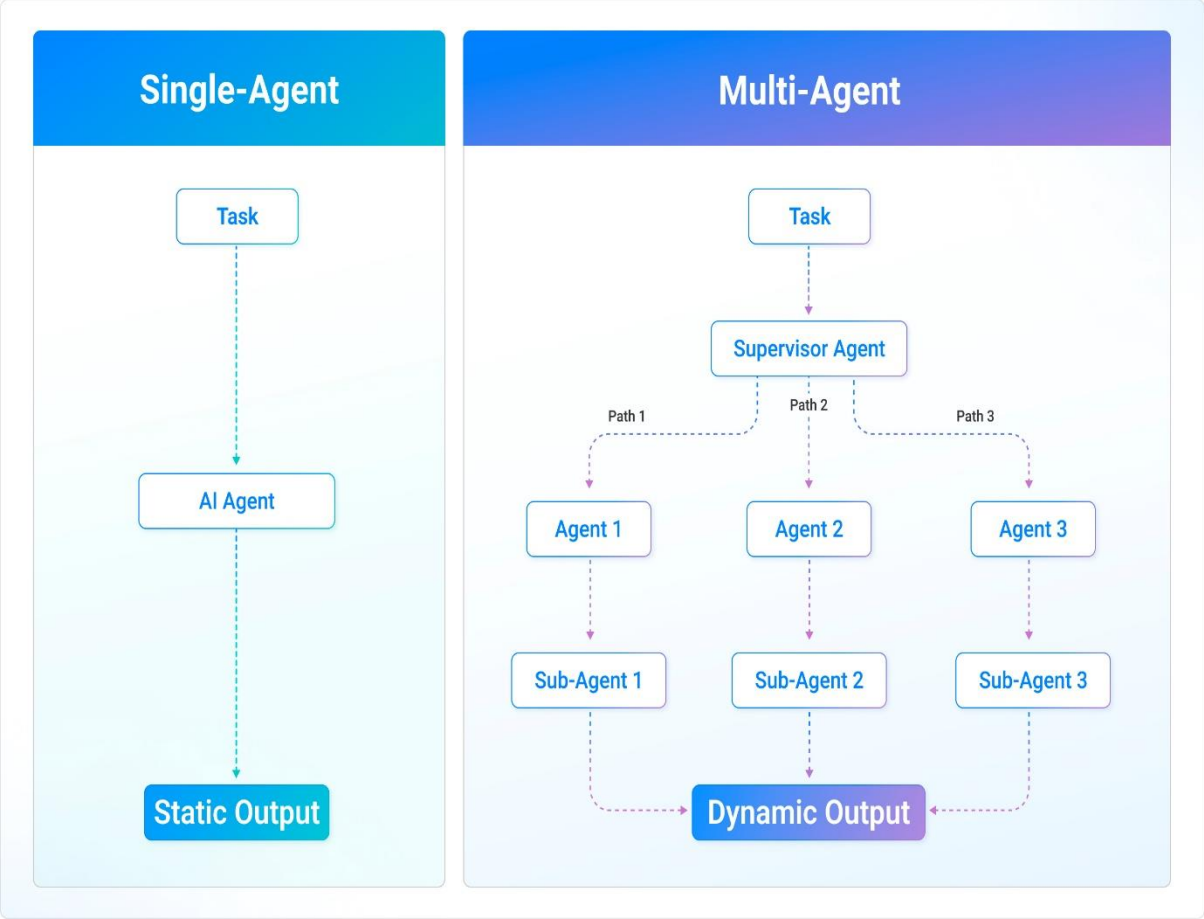
6.1. Performance metrics:

- Task completion time
- Throughput
- Resource utilization
- Scalability w.r.t number of agents
- Robustness to failures
- Communication overhead
- Cost (compute, energy)

6.2. When to choose SAS vs MAS:

Use SAS when tasks are tightly-coupled, small scale, easily modelled centrally, or need strong verifiability.

Use MAS when tasks naturally decompose into distributed subproblems, require redundancy, or require parallel/simultaneous operation.



Real-World Applications & Case Studies

7.1. Robotics & swarm systems:

From micro-robot swarms to multi-UAV teams — MAS provides robustness and parallelism. Stigmergy in search and map-building, leader-follower formations, and distributed coverage.

7.2. Autonomous vehicles and traffic management:

Vehicle platooning, intersection management, and routing benefit from MAS coordination among cars and infrastructure agents.

7.3. Smart grids & energy systems:

Distributed energy resources (DERs), demand response, and microgrid coordination rely on MAS for optimization and resilience.

7.4. Finance & algorithmic trading:

Multiple trading agents compete/cooperate; MAS models market microstructure and emergent phenomena.

7.5. Healthcare & distributed diagnostics:

Agents can represent diagnostic modules, sensors, and hospital nodes for distributed triage and resource allocation.

7.6. Logistics, warehousing, and supply chain:

Multi-robot warehouses (e.g., Kiva systems), distributed inventory agents, and route optimization.

7.7. Case Study 1: Swarm robotics for search & rescue:

Problem: rapidly search a collapsed building.

MAS approach: many small robots use local rules (coverage algorithm), share discovered map fragments, and collectively focus on promising areas.

Key metrics: coverage rate, time to first detection, fault tolerance.

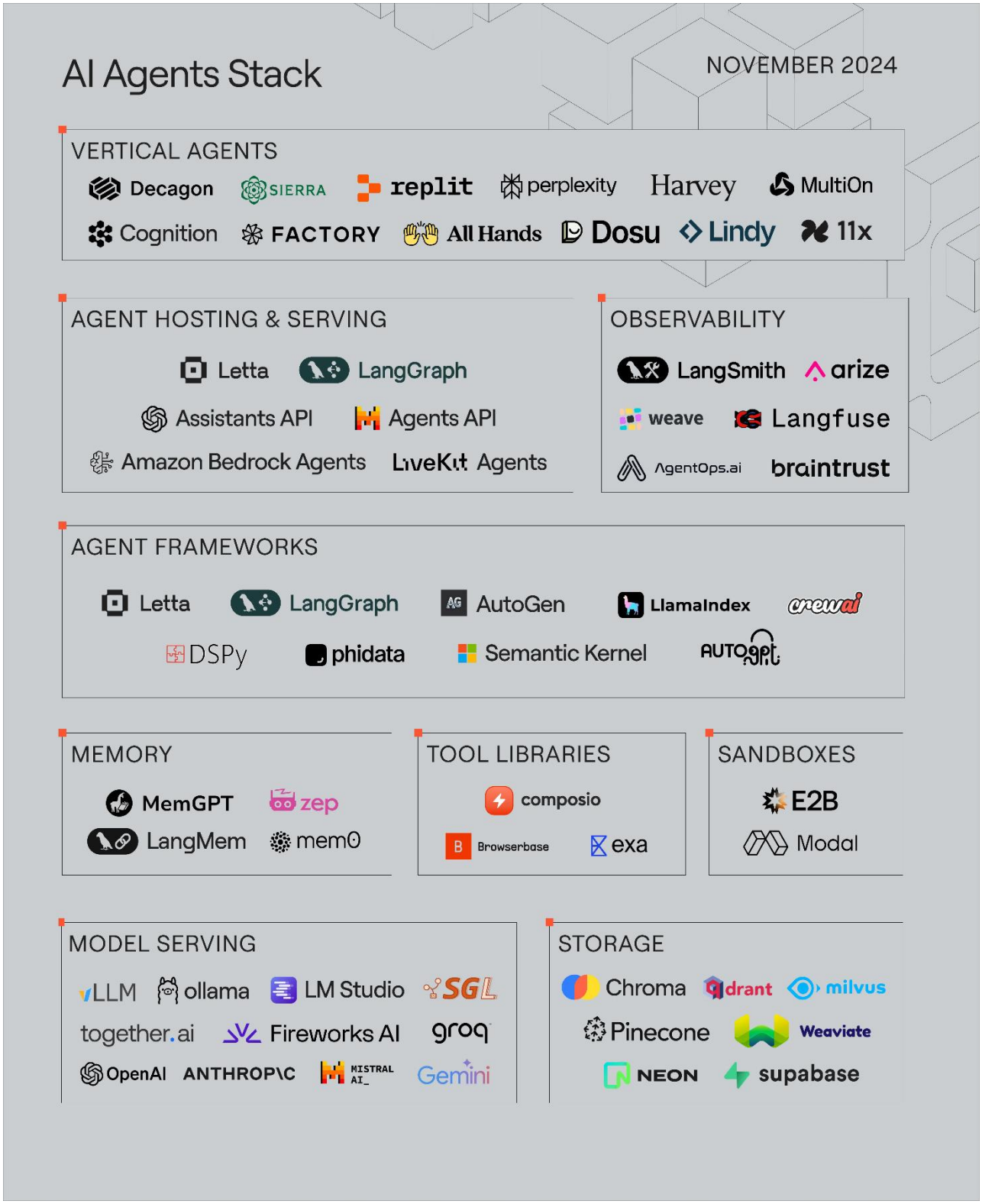
7.8. Case Study 2: Multi-agent traffic signal optimization:

Problem: reduce city congestion at intersections.

MAS approach: each junction is an agent that optimizes local signal timing using reinforcement learning and communicates with neighbouring junctions.

Key metrics: average travel time, queue lengths, fuel consumption.

AI Agent Stack



Implementation: Tools, Frameworks, and Datasets

8.1. Simulation frameworks:

ROS + Gazebo/Webots: robot simulations, sensor emulation.

NetLogo: educational MAS simulations, quick prototyping.

MASON: Java-based MAS simulation library for large-scale simulation.

SUMO: traffic simulation for vehicle-level MAS evaluation.

8.2. Agent frameworks:

JADE: Java Agent DEvelopment Framework (FIPA-compliant)

SPADE: Python-based agent framework

Custom stacks: using message buses (MQTT, ZeroMQ, ROS topics)

8.3. Testing & metrics:

- Fault injection
- Scalability testing by increasing agent count
- Network latency simulation
- Reproducible random seeds for experiments.

Ethical, Security, and Practical Considerations

9.1. Safety and verification:

Model checking, runtime verification, formal methods where applicable. For MAS, consider safe fallback behaviors and fail-safe central arbitration.

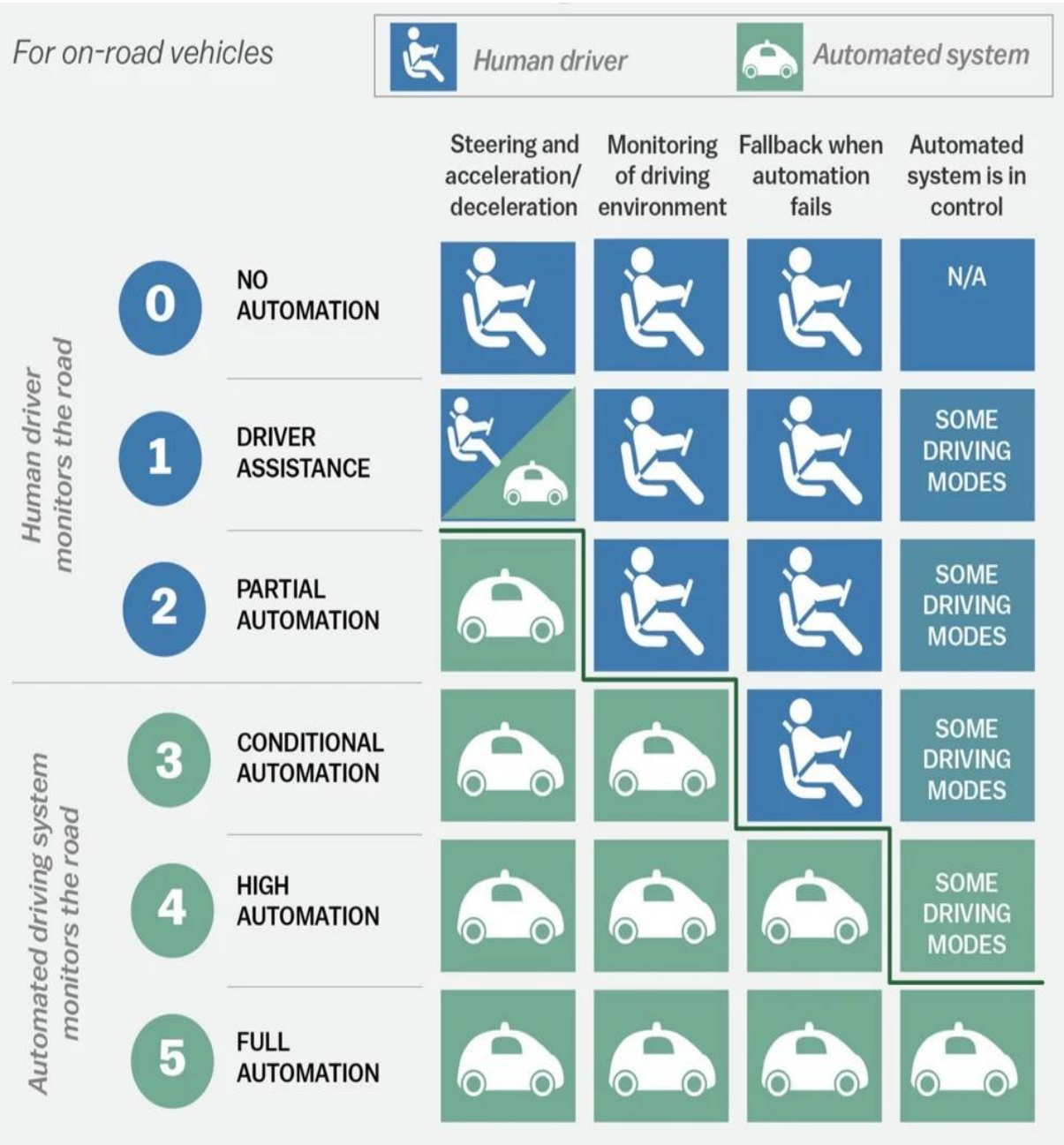
9.2. Privacy and data sharing:

When agents share data, minimize leakage via differential privacy, secure multiparty computation, or anonymization.

9.3. Adversarial agents and robustness:

Design for Byzantine faults, adversarial inputs, and sybil attacks in open MAS deployments.

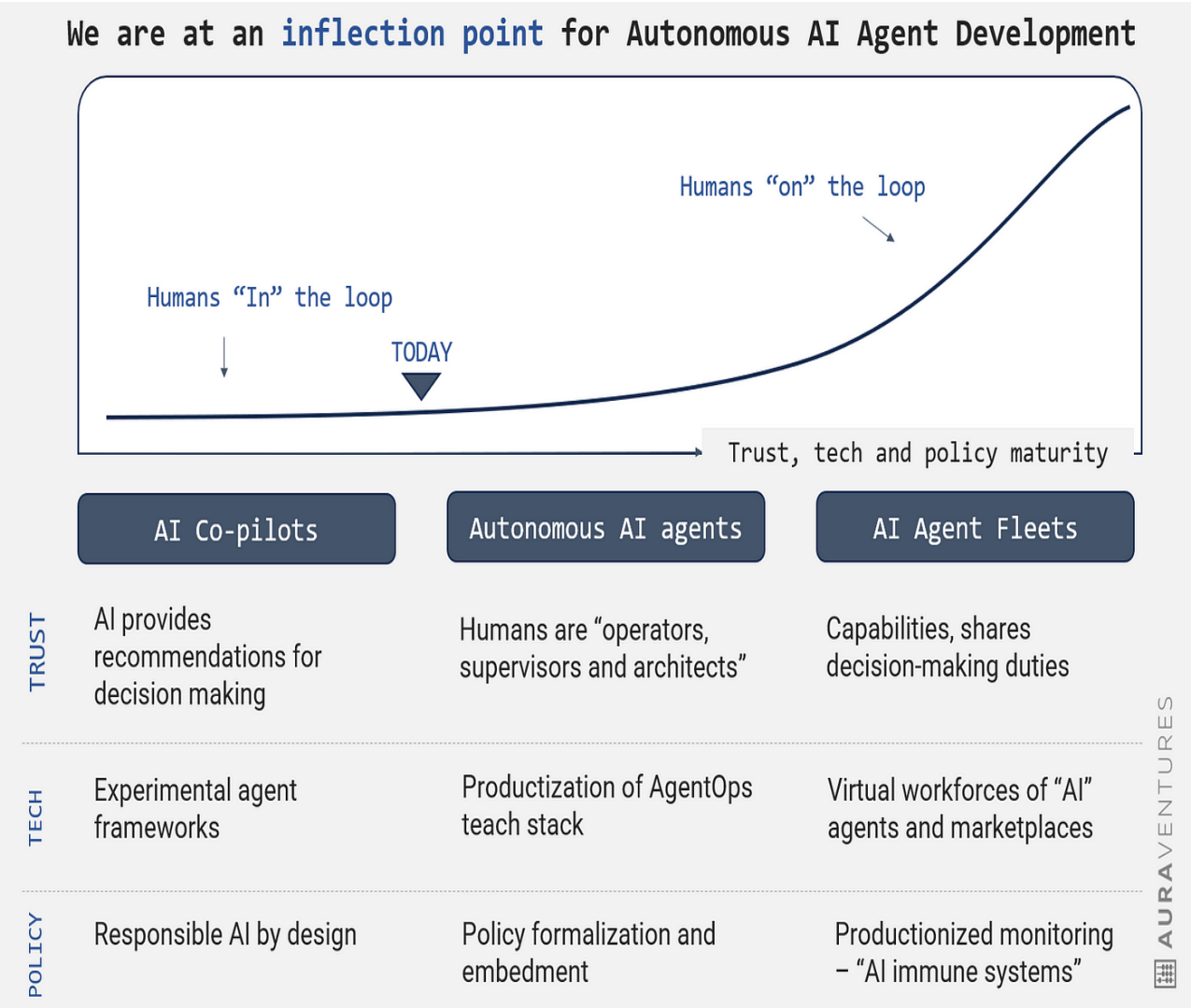
AI Agents through Industry POV



Conclusion and Future Directions

Summarize key findings:

SAS are simpler and verifiable; MAS offer scalability and robustness but are harder to predict and verify. Future work: hybrid verification methods, learning-stable MAS protocols, real-world deployments with human-in-the-loop control, and standards for safety certification.



Appendices

A. Glossary:

Define terms: agent, MAS, SAS, FIPA, ACL, stigmergy, auction, Nash equilibrium, consensus, Paxos/Raft.

B. Suggested reading & tool links:

Provide keywords for further searches (JADE agent framework, ROS Gazebo tutorials, NetLogo model library, SUMO traffic simulation examples, MASON library documentation).

C. Example code snippets and pseudo-code:

1) Pseudo-code — simple MAS bidding allocation

```
# pseudo-code: auction-based allocation

agents = [A1,A2,...]

items = [task1, task2,...]

for item in items:

bids = {}

for agent in agents:

bids[agent] = agent.compute_bid(item)

winner = argmax(bids)

assign(item, winner)
```

2) Pseudo-code — SAS planning loop

```
while not goal_reached:

observation = sense()

state = estimate_state(observation)

plan = planner.compute(state, goal)

execute(plan)

monitor()
```